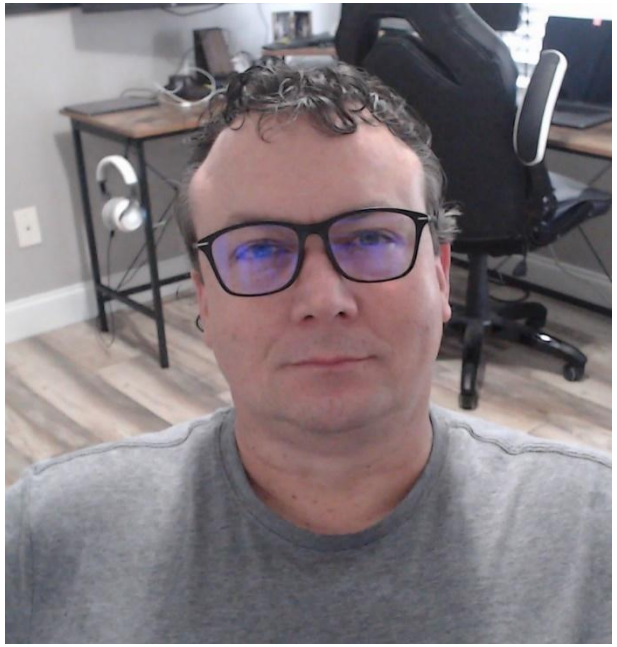
SQL Server

Storage, Memory, and Transaction Log Internals



Instructor: Brian Dietrick

SQL Server Series

1. Foundations and SSMS
2. Installation, Modeling, and Development
3. Core Administration and Operations
4. Performance Tuning and Deep Dive

1. Foundations and SSMS

- Day 1 – Platform and Instance Foundations
- Day 2 – Storage, Memory, and Transaction Log Internals
- Day 3 – SSMS and Observability Tooling
- Day 4 – Security and Support T-SQL Essentials

ECU

Day 2 Agenda

- Storage hierarchy and allocation behavior
- Pages, extents, and file-level implications
- Buffer pool, plan cache, checkpoint, lazy writer
- Transaction log and write-ahead logging
- Guided demo: 7-step walk-through
- Deep lab and troubleshooting review

ECU

Learning Objectives

- Explain pages, extents, and allocation maps
- Distinguish data files vs. log files
- Describe buffer pool vs. plan cache
- Compare lazy writer vs. checkpoint
- Explain write-ahead logging and rollback behavior
- Run supported inspection queries safely

Teaching Flow and Timeboxes

- Segment 1: Storage fundamentals
- Segment 2: Allocation behavior and files
- Segment 3: Memory internals for operations
- Segment 4: Log internals and WAL
- Segment: Guided walkthrough + knowledge check

Why This Module Matters for Support

- Many incidents are symptom-layered
- Object growth and file growth are not identical
- Memory pressure and plan issues are often conflated
- Log growth is frequently misdiagnosed

Storage Hierarchy Mental Model

- SQL Server instance hosts one or more databases
- Databases contain tables and indexes
- Data persists in MDF/NDF files
- Files are made of pages and extents

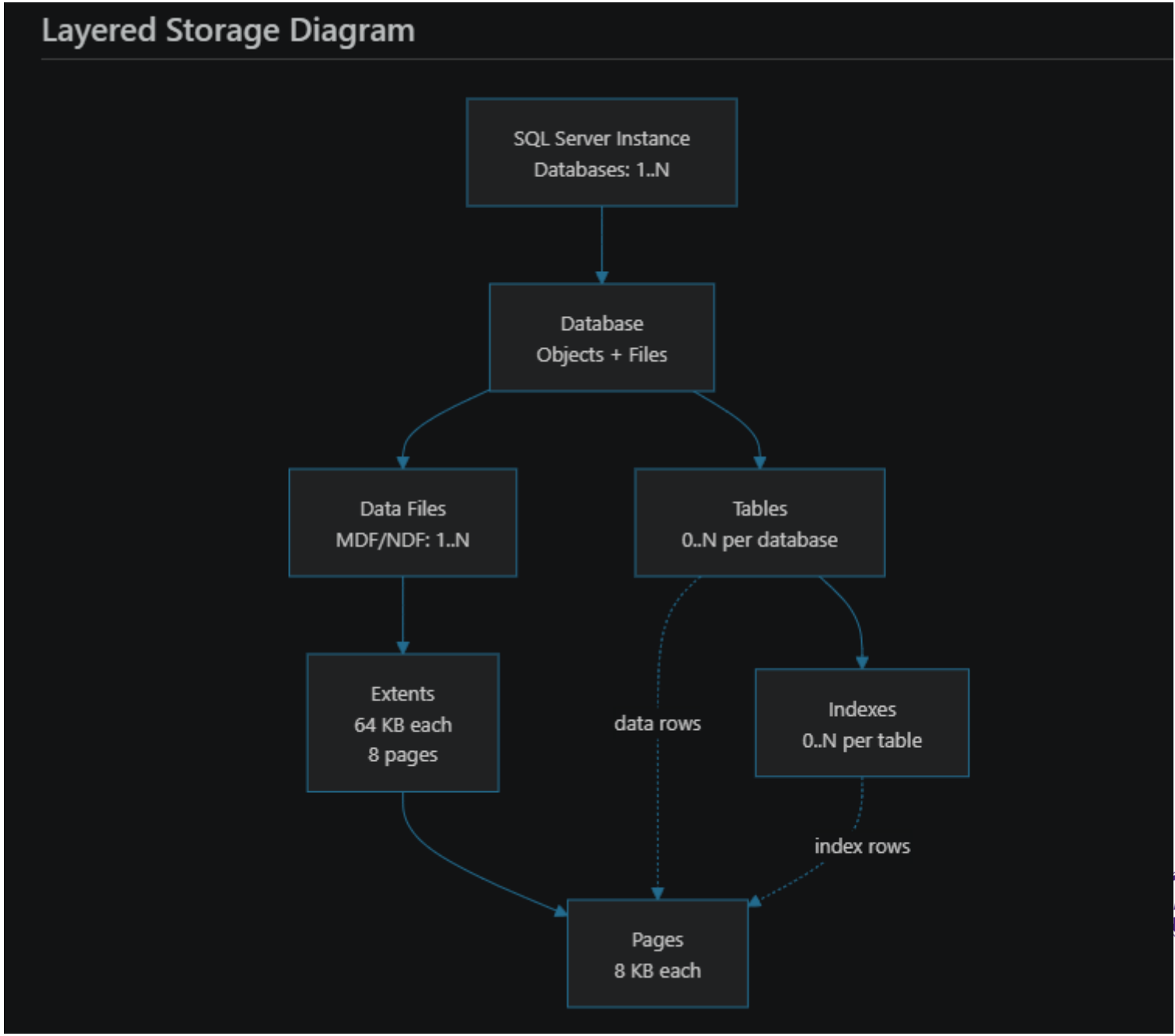
Layered Storage Diagram

SQL Server Instance → Databases → Data Files → Extents → Pages

SQL Server Instance → Databases → Tables → Indexes → Pages

SQL Server Instance → Databases → Objects → Pages

Layered Storage Diagram



ECU

Size and Count Notation Quick Reference

- 1 page = 8 KB
- 1 extent = 8 pages = 64 KB
- 1 MB = 128 pages = 16 extents
- 1 GB = 131072 pages = 16384 extents

Page Structure at a High Level

- Page header (96 bytes)
- Data/index rows
- Free space region
- Slot array for row offsets

Extents and Allocation Behavior

- SQL Server allocates space in extent-sized chunks
- Extents unify page management at scale
- Object growth eventually means more extents
- Extent use affects file growth patterns

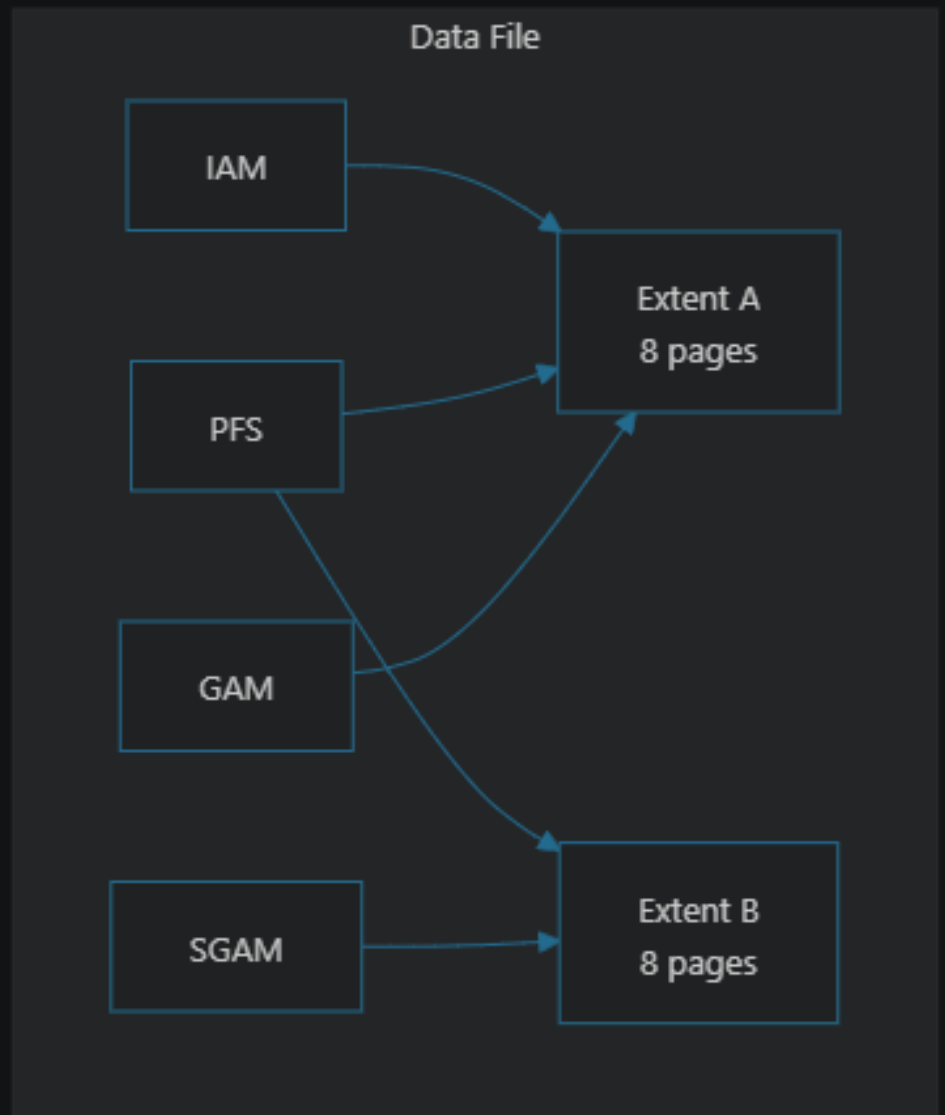
Allocation Maps: Purpose First

- Allocation maps are bookkeeping pages
- They answer: free, used, and owned space questions
- They explain why file size and used data can diverge

Allocation Maps You Should Recognize

- GAM: tracks free extents
- SGAM: tracks mixed extents with free pages
- PFS: tracks free space per page
- IAM: tracks extents owned by an object

Allocation Map Relationship Diagram



Data Files vs. Log Files

- Data file: current object state (tables/indexes)
- Log file: ordered change history
- Different roles, different growth drivers
- Different troubleshooting starting points

Symptom Triage by File Type

- Inserts fail, data file full -> capacity/object growth path
- Inserts fail, log full -> reuse/backup/recovery path
- Same error text can hide different root causes

Mini Exercise: Storage/File Interpretation

- Given file size output and table growth note:
- Choose first follow-up focus:
  - table design
  - data file capacity
  - log file capacity

Mini Exercise: Reserved vs. Used Space

- Why can reserved space exceed row data?
- Explain two causes from allocation perspective
- Distinguish object reservation from row payload

Memory Fundamentals Overview

- Buffer pool: data/index page cache
- Plan cache: compiled query plans
- Lazy writer: reusable buffer availability
- Checkpoint: dirty page flushing for recovery

Buffer Pool Essentials

- Caches data and index pages
- Reduces repeated physical reads
- Improves response time for hot data
- Does not replace durable files

Plan Cache Essentials

- Stores compiled execution plans
- Reduces repeated compilation CPU
- Plans can age out or be invalidated
- Stores plans, not table data

Lazy Writer Explained

- Background process for reusable buffers
- Triggered by memory availability needs
- May write dirty pages while freeing buffers
- Not the transaction durability checkpoint

Checkpoint Explained

- Writes dirty data pages to data files
- Reduces crash recovery workload
- Supports recovery objectives
- Does not replace transaction log durability

Lazy Writer vs. Checkpoint

- Lazy writer: buffer availability focus
- Checkpoint: recovery workload focus
- Both may write dirty pages
- Different triggers and operational intent

Transaction Log Fundamentals

- Ordered, durable record of changes
- Critical for recovery correctness
- Required for many write operations
- Central to commit and rollback behavior

Write-Ahead Logging (WAL)

- Log records are persisted before durable data-page state
- Supports redo/undo after interruption
- Enables consistent crash recovery

Commit vs. Rollback Practical View

- Commit durability is log-driven
- Data pages may flush later
- Rollback is active undo work
- Large rollback can take noticeable time

Log Reuse and File Size

- Reuse eligibility != physical shrink
- Reuse depends on recovery context
- Active transactions/backups can delay reuse
- SQL Server 2016 note: truncation and shrinking are different operations

Guided Demo Overview (7 Steps)

- Step 1: file layout inspection
- Step 2: allocation-level space usage
- Step 3: sp_spaceused summary
- Step 4: buffer pool page presence
- Step 5: plan cache and counters
- Step 6: explicit transaction + rollback
- Step 7: log reuse status check

Demo Setup Summary

- Create Module2TrainingDemo database
- Inspect initial file metadata
- Create dbo.StorageDemo table
- Seed sample rows

Demo Step 1 and Step 2 Outcomes

- Step 1: identify ROWS vs LOG file roles
- Step 2: compare reserved, used, and data KB
- Understand file-level vs object-level views

Demo Step 3 and Step 4 Outcomes

- Step 3: use sp_spaceused as concise summary
- Step 4: observe cached pages for StorageDemo
- Connect reads to buffer pool behavior

Demo Step 5 Outcomes

- Detect potential plan reuse signals (usecounts)
- Read selected Buffer Manager counters
- Differentiate checkpoint pages/sec from lazy writes/sec

Demo Step 6 and Step 7 Outcomes

- Observe in-transaction values then rollback reversion
- Confirm rollback restores prior committed state
- Query log_reuse_wait_desc from sys.databases

Deep Lab Goal and Prerequisites

- Goal: connect storage, memory, and log evidence end-to-end
- Requires Module2TrainingDemo setup
- VIEW SERVER STATE needed for buffer descriptors
- Prefer two SSMS windows for transaction visibility task

Deep Lab Task Flow

- Task 1: inspect files
- Task 2: compare allocation vs summary space
- Task 3: inspect cached pages
- Task 4: inspect plan cache + counters
- Task 5: two-session rollback visibility
- Task 6: analyze log reuse state

Troubleshooting Playbook

- Separate data-file vs log-file growth causes
- Distinguish checkpoint from lazy writer
- Evaluate log reuse before shrink decisions
- Interpret summary and detailed views together
- Treat DMV permission limits as scope boundaries

Knowledge Check (Questions)

- Page and extent sizes?
- Why allocation maps matter?
- Why reserved can exceed used?
- Buffer pool vs plan cache purpose?
- Checkpoint vs lazy writer purpose?
- WAL durability ordering?
- Why rollback can take time?
- Why inspect log_reuse_wait_desc first?

Knowledge Check (Answer Summary)

- Page: 8 KB; extent: 8 pages (64 KB)
- Allocation maps track free/used/owned space
- Reserved can exceed used due to allocation state
- Buffer pool caches pages; plan cache caches plans
- Checkpoint supports recovery; lazy writer supports reusable buffers
- WAL: log first, then durable data-page progression
- Rollback is active undo work, not instant erase
- Reuse reason should be diagnosed before shrink actions

SQL Server 2016 Notes and Boundaries

- SQL Server 2016 note: log truncation and shrink are different
- SQL Server 2016 note: server-scoped DMVs require VIEW SERVER STATE
- SQL Server 2016 note: use documented metadata views for support workflows

Wrap-Up and Transition

- You can now map symptoms to storage, memory, or log layers
- You can run supported queries safely and interpret outputs
- Next module: SSMS and observability tooling workflows